FROM HIDDEN SPIKES TO WHOLE-NETWORK SAVINGS: REGULARIZATION VERSUS THRESHOLD SCALING IN A SPIKING NEURAL NETWORK

PREPRINT

Saksham Gupta
Department of Electrical and Electronics Engineering
Birla Institute of Technology and Science (BITS), Pilani,
Hyderabad Campus, Hyderabad, India
f20250785@hyderabad.bits-pilani.ac.in

ABSTRACT

Suppressing hidden spikes need not substantially reduce the synaptic activity of a whole network. We compare hidden-spike regularization during training with uniform hidden-threshold scaling after training in a rate-encoded Fashion-MNIST network. An original weak-penalty primary comparison was inconclusive for total effective synaptic operations (SynOps). We then fixed a stronger-penalty primary comparison before training 20 new models: two simulation windows, two penalties, and five paired training seeds. Three input encodings were averaged within each seed. At 16 steps, regularization reduced total SynOps by 3.449% (95% paired interval 3.150–3.748%) and changed accuracy by +0.046 percentage points (-0.437 to $+0.529$), meeting the declared one-point accuracy criterion. Hidden-output SynOps fell by 43.585%. Against the validation-selected threshold policy, regularization achieved 1.024 points higher accuracy (0.444–1.604) and 2.742% fewer total SynOps (2.428–3.056%). Unchanged input events impose an accounting ceiling averaging 7.939% total savings at 16 steps, even if hidden-output operations vanished. The results support regularization under this architecture and training budget; five seeds and dependence on the interval model limit statistical confidence. They do not establish hardware energy savings or generalization to another benchmark.

Keywords spiking neural networks · activity regularization · threshold scaling · synaptic operations · neuromorphic computing

1 Introduction

A conventional artificial neural network passes numerical activations between layers. A spiking neural network (SNN) communicates through brief binary events called spikes. Each neuron accumulates incoming signals, lets its internal state decay, and emits a spike when that state exceeds a threshold. Neuromorphic hardware can exploit sparse events, but a software simulation on a GPU does not automatically gain the same benefit. Surrogate-gradient learning supplies an approximate derivative for the discrete spike decision, making SNNs trainable with gradient-based optimization [1]. Practical tools such as snnTorch implement these dynamics within familiar learning workflows [2].

Our question is whether a simple penalty on hidden activity can reduce whole-network synaptic operations while keeping accuracy within one percentage point of baseline, and whether it offers a better operating point than raising hidden thresholds after training. A synaptic operation (SynOp) here means one active source event passed through one nonzero connection. Suppressing a spike with many outgoing connections saves more counted operations than suppressing one with few connections. Input spikes matter too.

1.1 Related work and study contribution

Activity regularization is an established approach. Narduzzi and colleagues compared several activation penalties before converting conventional networks into SNNs, observing substantial reductions in spikes and SynOps [3]. Yan and colleagues combined spike regularization with synaptic rewiring [4]. Suetake and colleagues formulated a fanout-weighted synaptic interaction penalty and distinguished its operation-based objective from unweighted or layer-balanced firing penalties [5]. These studies motivate reporting operation counts separately from firing rates.

Recent methods extend the intervention beyond a fixed occupancy penalty. UniSparse, published at ICML 2026, couples spike sparsification through batch-normalization parameters with weight pruning under an operation-based constraint [6]. Its trainable, architecture-dependent intervention differs from the uniform post-training threshold control tested here. Temporal coding offers another route: Stanojevic and colleagues analyze time-to-first-spike networks, in which the timing of a sparse event carries information [7]. Their results cannot be compared directly with repeated Bernoulli rate encoding. Kim and colleagues also show that rate and direct encoding trade accuracy, robustness, and implementation-dependent energy costs [8].

Our contribution is a deliberately narrow empirical comparison: matched architecture and training budget, validation-locked policies, paired seed-level uncertainty, and a decomposition linking hidden-output savings to the unchanged input cost. We use established regularization and accounting principles; the added evidence is their controlled application and fresh-seed replication in this setting. No state-of-the-art classification claim follows from these experiments. NeuroBench's distinction between algorithmic complexity and measured system performance guides our interpretation of SynOps [9].

The original weak-penalty primary comparison was inconclusive for operation savings. Stronger exploratory results motivated a locally frozen 20-model cohort with fresh seeds. We retain the original finding and analyze the cohorts separately. A further accounting analysis explains why apparently large hidden-layer improvements can produce only modest total savings.

2 Methods

2.1 Data and stochastic input encoding

Fashion-MNIST contains ten classes of 28×28 grayscale clothing images, with 60,000 training and 10,000 test examples [10]. A class-stratified split, fixed with seed 2026, allocated 54,000 examples to training and 6,000 to validation. We flattened images into 784 intensities in $[0, 1]$ by dividing pixel values by 255. No augmentation was used. In rate encoding, a brighter pixel has a higher probability of producing a spike at each step; the image is therefore presented as a short random sequence of binary patterns.

For image n, pixel j, and simulation step t, Bernoulli rate encoding produces

$$s_{ntj}^{(0)} = \mathbf{1}[U_{ntj} < x_{nj}], \qquad U_{ntj} \sim \mathrm{Uniform}(0, 1), \tag{1}$$

where x_{nj} is normalized intensity, $s_{ntj}^{(0)}$ is a binary input event, and $\mathbf{1}[\cdot]$ is the indicator function. Random numbers are generated in example-major banks of shape (examples, maximum steps, 784); a shorter window uses a prefix. Image order and encoding seed rules are matched across interventions within a cohort.

The original cohort used maximum window 32, validation encoding seeds 101–103, and test seeds 201–203. The fresh cohort used maximum window 16, validation seeds 301–303, and test seeds 401–403. Changing the maximum bank length changes random-number allocation across images, including during training. Thus the cohorts are not pooled. The data split and test images are identical across cohorts. Test examples were excluded from optimization and policy selection, but this benchmark had been inspected before the fresh-cohort design was fixed.

2.2 Network dynamics and learning objective

The fully connected network has widths $N_0 = 784$, $N_1 = 128$, $N_2 = 64$, and $N_3 = 10$, with learned biases and leaky integrate-and-fire neurons in layers $l = 1, 2, 3$. The two middle layers are hidden layers: they transform the input before the ten output neurons represent the clothing classes. For one image, the vector dynamics are

$$\boldsymbol{v}_t^{(l)} = \beta \boldsymbol{u}_{t-1}^{(l)} + W^{(l)} \boldsymbol{s}_t^{(l-1)} + \boldsymbol{b}^{(l)}, \tag{2}$$

$$\boldsymbol{s}_t^{(l)} = \mathbf{1}[\boldsymbol{v}_t^{(l)} > \theta_l], \qquad \boldsymbol{u}_t^{(l)} = \boldsymbol{v}_t^{(l)} - \theta_l \,\mathrm{stopgrad}(\boldsymbol{s}_t^{(l)}). \tag{3}$$

Here $\boldsymbol{v}_t^{(l)}$ is pre-reset voltage, $\boldsymbol{u}_t^{(l)}$ is post-reset membrane state, $W^{(l)}$ maps the preceding layer to layer l, and $\boldsymbol{b}^{(l)}$ is its bias. Comparisons act elementwise. We use leak $\beta = 0.9$ and baseline threshold $\theta_0 = 1$. States begin at zero for each presentation. Reset is immediate and subtractive; $\mathrm{stopgrad}$ retains the forward spike value while detaching its contribution to the reset gradient.

The forward spike decision remains the hard indicator in Eq. (3). In backpropagation, its derivative with respect to the threshold-relative voltage $q = v - \theta_l$ is replaced by the arctangent surrogate derivative

$$g_\alpha(q) = \frac{1}{\pi}\arctan\Big(\frac{\pi\alpha q}{2}\Big) + \frac{1}{2}, \qquad g'_\alpha(q) = \frac{\alpha/2}{1 + (\pi\alpha q/2)^2}, \qquad \alpha = 2. \tag{4}$$

The scalar α controls surrogate slope; it is distinct from the threshold multiplier defined below.

For batch item b and class k, logits are $z_{bk} = \gamma T^{-1} \sum_{t=1}^{T} s_{btk}^{(3)}$, where T is the number of simulation steps and $\gamma = 10$. Class probabilities and cross-entropy are

$$p_{bk} = \frac{\exp(z_{bk})}{\sum_{h=1}^{10} \exp(z_{bh})}, \qquad \mathcal{L}_{\mathrm{CE}} = -\frac{1}{B}\sum_{b=1}^{B} \log p_{b,y_b}, \tag{5}$$

with batch size B and target class y_b. Prediction uses the largest logit; ties use the lowest class index. Hidden occupancy and the training objective are

$$\rho_l = \frac{1}{BTN_l}\sum_{b=1}^{B}\sum_{t=1}^{T}\sum_{j=1}^{N_l} s_{btj}^{(l)}, \qquad \mathcal{L} = \mathcal{L}_{\mathrm{CE}} + \lambda\frac{\rho_1 + \rho_2}{2}. \tag{6}$$

Here ρ_l is the fraction of hidden neuron–time entries that spike, and λ is the penalty coefficient. The two hidden layers receive equal weight. This penalty excludes input and output occupancy and does not directly minimize fanout-weighted SynOps.

Adam used learning rate 0.001, batches of 128, and 15 epochs; evaluation batches contained 256 examples. Each model retained the earliest epoch attaining the highest mean validation accuracy across three encodings, replacing a checkpoint only on strict improvement. Matched training seeds shared initialization and shuffle/encoding seed rules. Deterministic execution was requested, and software versions and checkpoint hashes were recorded.

2.3 Cohorts and threshold calibration

The original cohort crossed $T \in \{8, 16, 32\}$, $\lambda \in \{0, 0.1, 1\}$, and seeds 11, 22, 33, 44, 55: 45 SNNs. Five matching-width ReLU networks provided an ANN reference, giving 50 trained models. The original primary contrast was $\lambda = 0.1$ versus 0 at $T = 16$ and unchanged thresholds. The stronger-penalty comparisons were secondary. A later checkpoint-only follow-up refined the threshold grid after the original test benchmark had been inspected.

The fresh cohort contained exactly 20 new SNNs: $T \in \{8, 16\}$, $\lambda \in \{0, 1\}$, and seeds 1011, 1022, 1033, 1044, 1055. No original model or ANN was retrained. Its fixed primary contrast was $\lambda = 1$ versus 0 at $T = 16$ and unchanged thresholds. The protocol and execution plan were hashed before training, with five seeds and no significance-dependent extension. This was a local protocol freeze, not an external preregistration. Training and evaluation used two Kaggle T4 GPUs with one worker per device.

Threshold scaling reused unregularized weights without further training:

$$\theta_l(a) = \begin{cases} a\theta_0, & l \in \{1, 2\}, \\ \theta_0, & l = 3, \end{cases} \qquad a \in \{1.00, 1.05, \ldots, 1.50\}. \tag{7}$$

The scalar a is shared by both hidden layers; the output threshold remains 1. At each window, we selected one policy across all five seeds for each strategy. Let $c = (\lambda, a)$ denote a candidate, $c_0 = (0, 1)$ the baseline, $\bar{A}_{\mathrm{val}}(c)$ mean validation accuracy as a fraction, and $\bar{K}_{\mathrm{val}}(c)$ mean total SynOps. The rule was

$$\mathcal{F} = \left\{ c \in \mathcal{C} : \bar{A}_{\mathrm{val}}(c) \geq \bar{A}_{\mathrm{val}}(c_0) - \frac{\delta}{100} \right\}, \qquad c^* = \arg\min_{c \in \mathcal{F}} \bar{K}_{\mathrm{val}}(c), \qquad \delta = 1. \tag{8}$$

Here δ is the accuracy allowance in percentage points. The candidate set $\mathcal{C}$ comprised trained penalties at $a = 1$ for regularization, or the 11 factors in Eq. (7) at $\lambda = 0$ for scaling. Ties favored higher accuracy, then lower λ, then lower a. Validation means averaged encodings within seeds, then seeds. The selection record was locked before test evaluation. This point-estimate selection rule is distinct from the interval-based test criterion below.

The fresh evaluation grid contained 120 model–policy cells per split: ten baseline models evaluated at 11 factors and ten regularized models at $a = 1$. Three encodings per cell gave 360 validation and 360 test records. These are evaluations of 20 trained models, not 120 independent training runs. Figure 1 summarizes the workflow.

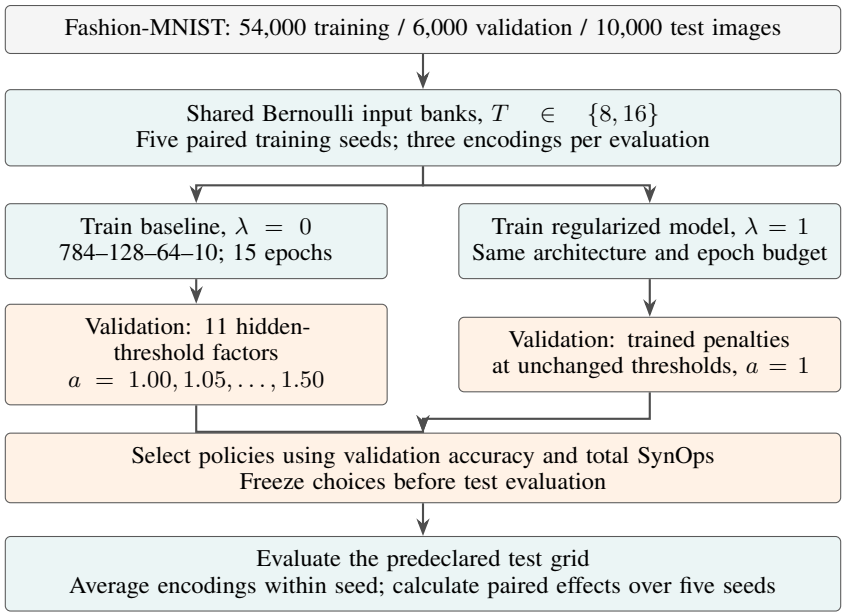


Figure 1: Fresh-seed study workflow. Threshold scaling reuses baseline weights; regularization changes training. The primary comparison fixes $T = 16$, $\lambda = 1$ versus 0, and $a = 1$ before training. Validation-selected policies are secondary. The diagram summarizes 20 trained models, not additional training for each threshold factor.

2.4 Effective synaptic operation accounting

For source neuron j in layer $l-1$, define nonzero outgoing fanout $f_j^{(l)} = \sum_{k=1}^{N_l} \mathbf{1}[W_{kj}^{(l)} \neq 0]$. The per-image operation count in receiving layer l is

$$K_l = \sum_{t=1}^{T} \sum_{j=1}^{N_{l-1}} s_{tj}^{(l-1)} f_j^{(l)}, \qquad K = \sum_{l=1}^{3} K_l. \tag{9}$$

All archived checkpoints had dense nonzero weights. Writing $S_l = \sum_{t,j} s_{tj}^{(l)}$ for the spike total in source layer l over one presentation gives

$$K = 128S_0 + 64S_1 + 10S_2, \qquad K_{\text{hid}} = 64S_1 + 10S_2. \tag{10}$$

The hidden-output component K_{hid} counts operations driven by hidden spikes; it is not an unweighted hidden-spike count. Output spikes have no outgoing layer and contribute no SynOps here. Biases, membrane updates, routing, and memory traffic are excluded. A dense GPU simulator can execute the same matrix operations despite fewer effective source events. These counts therefore do not measure GPU work, device latency, or energy.

2.5 A ceiling imposed by unchanged input operations

Let I_i be the mean input-to-hidden SynOps for seed i, and let H_i^B and H_i^M be hidden-output SynOps for baseline B and intervention M. When input events and input fanout are unchanged, $K_i^B = I_i + H_i^B$ and $K_i^M = I_i + H_i^M$. Define baseline input share $p_i = I_i / K_i^B$ and fractional hidden reduction $r_i^H = 1 - H_i^M / H_i^B$. Then

$$r_i^K = 1 - \frac{K_i^M}{K_i^B} = (1 - p_i) r_i^H \leq 1 - p_i, \tag{11}$$

where r_i^K is fractional total reduction and the bound uses $H_i^M \geq 0$. Multiplying a fraction by 100 expresses a percentage. This is an accounting identity, not a new learning theorem: eliminating all hidden-output operations could save at most the baseline hidden share. Such elimination need not preserve accuracy. The identity applies per seed; products of averages generally differ from averages of products. We added this explanatory analysis after observing the replication results, without changing the frozen primary test.

2.6 Paired analysis and evidence audit

The independent units are five paired training-seed means. For method m, training seed i, encoding r, and test image n, let $\hat{y}^m_{nir}$ be the prediction, y_n its label, and K^m_{nir} its total SynOps. With $R = 3$ encodings and $N = 10{,}000$ test images,

$$A^m_i = \frac{1}{RN}\sum_{r=1}^{R}\sum_{n=1}^{N}\mathbf{1}[\hat{y}^m_{nir} = y_n], \qquad K^m_i = \frac{1}{RN}\sum_{r=1}^{R}\sum_{n=1}^{N}K^m_{nir}. \tag{12}$$

For a comparison of method X with reference Y, define

$$d^A_i = 100(A^X_i - A^Y_i), \qquad d^K_i = 100\left(1 - \frac{K^X_i}{K^Y_i}\right). \tag{13}$$

Accuracy effects d^A_i are percentage points; operation reductions d^K_i are percentages, positive when X uses fewer operations. Hidden-output reductions replace K by K_{hid}. Averaging per-seed ratios differs from taking a ratio of group means.

For either effect d, the mean $\bar{d}$, sample standard deviation s_d, and two-sided 95% interval are

$$\bar{d} = \frac{1}{5}\sum_{i=1}^{5} d_i, \quad s_d = \sqrt{\frac{1}{4}\sum_{i=1}^{5}(d_i - \bar{d})^2}, \quad [L_d, U_d] = \bar{d} \pm t_{0.975,4}\frac{s_d}{\sqrt{5}}, \quad t_{0.975,4} = 2.776445. \tag{14}$$

For a baseline-relative contrast, success requires both $L_{d^A} > -\delta$ and $L_{d^K} > 0$. This conservative interval rule assesses retention within the stated allowance; an interval crossing zero for accuracy is not proof of equality. Intervals use an approximate seed-level Student t model. They are unadjusted, conditional on the fixed split and encoding banks, and do not include uncertainty from policy selection. Three encodings per seed do not create 15 independent training replicates. Here a percentage point is an absolute accuracy difference: a change from 88% to 87% is a one-point loss.

The returned archive was checked against its hash inventory and frozen protocol. We verified all 20 checkpoint identities, epoch histories, validation choices, and 720 raw evaluation records; reconstructed accuracy, confusion matrices, and dense-fanout counts from saved arrays; and regenerated input-event counts from the verified dataset and encoding seeds. Final summaries use float64 aggregation of raw evidence. Native float32 summaries agree at the reported three-decimal precision. This is a consistency audit, not an independent rerun of network outputs.

3 Results

3.1 The fixed fresh-seed primary contrast meets both criteria

At $T = 16$, $\lambda = 1$ reduced total SynOps by 3.449% (95% interval 3.150–3.748%) relative to baseline. Accuracy changed by +0.046 points (-0.437 to $+0.529$). The lower accuracy bound exceeded -1 point and the lower operation-reduction bound exceeded zero, satisfying both declared criteria. Mean accuracy was 87.640% for baseline and 87.686% for regularization. The data support retention within the allowance; they do not establish improved accuracy over baseline.

The $T = 8$ supporting contrast also met both criteria (Table 1). Its total-operation reduction was 5.132% (4.349–5.915%), with accuracy change -0.061 points (-0.373 to $+0.250$). Figure 2 shows individual paired seed effects as well as their uncertainty summaries. The two fresh-cohort windows are not additional independent replications of the primary contrast.

Table 1: Fresh-cohort regularization ($\lambda = 1$, $a = 1$) versus unmodified baseline. Entries are paired means [unadjusted 95% t intervals] over five training-seed means, each averaging three encodings. Positive effects favor regularization. The $T = 16$ row is the fixed primary contrast.

T	Accuracy change (pp)	Total SynOps reduction (%)	Hidden-output SynOps reduction (%)
8	-0.061 [-0.373, 0.250]	5.132 [4.349, 5.915]	63.424 [59.812, 67.037]
16	0.046 [-0.437, 0.529]	3.449 [3.150, 3.748]	43.585 [38.159, 49.011]

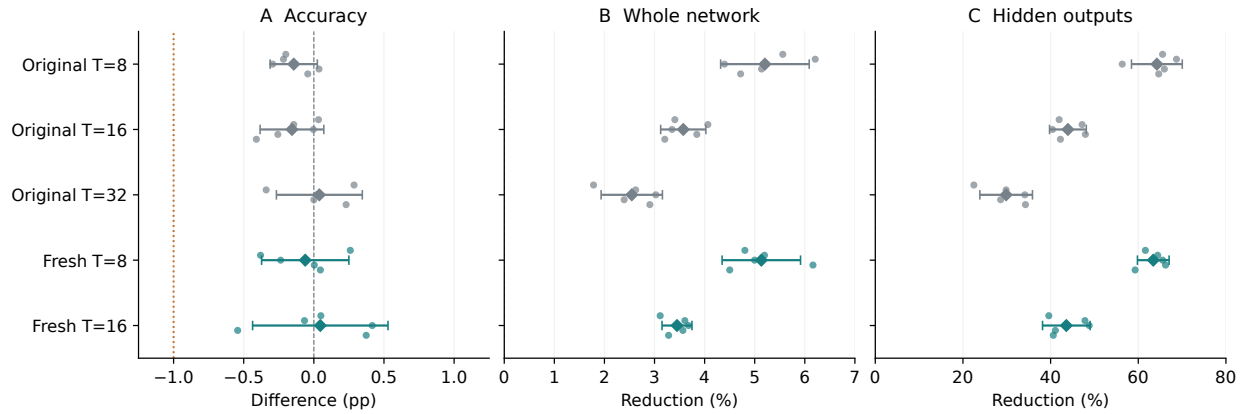


Figure 2: Paired regularization effects across cohorts. Dots represent five training-seed effects; diamonds and horizontal bars show their means and unadjusted 95% paired t intervals. Each seed first averages three encodings. All contrasts use $\lambda = 1$ versus baseline at $a = 1$. The dotted line marks the -1-point accuracy criterion. Original-cohort results are secondary; fresh $T = 16$ is the fixed replication primary. Cohorts are analyzed separately.

3.2 Regularization improves on the selected threshold operating points

Validation selected $\lambda = 1$ at both fresh-cohort windows, and threshold factors $a = 1.30$ at $T = 8$ and $a = 1.35$ at $T = 16$. Relative to the selected threshold policy at 16 steps, regularization achieved 1.024 points higher accuracy (0.444–1.604) and 2.742% fewer total SynOps (2.428–3.056%). At eight steps, the corresponding advantages were 0.936 points (0.239–1.633) and 4.496% fewer total SynOps (3.802–5.191%; Table 2). These secondary intervals exclude zero for both outcomes.

Table 2: Fresh-cohort regularization versus validation-selected threshold scaling. Selected factors are $a = 1.30$ at $T = 8$ and $a = 1.35$ at $T = 16$. Effects and intervals follow Table 1; these are secondary comparisons.

T	Accuracy change (pp)	Total SynOps reduction (%)	Hidden-output SynOps reduction (%)
8	0.936 [0.239, 1.633]	4.496 [3.802, 5.191]	60.148 [56.326, 63.970]
16	1.024 [0.444, 1.604]	2.742 [2.428, 3.056]	37.903 [31.950, 43.856]

The selected threshold policies reduced total SynOps by 0.666% at eight steps and 0.726% at 16 steps. Their baseline-relative accuracy changes were -0.997 points (-1.709 to -0.285) and -0.978 points (-1.782 to -0.174), respectively. Although the mean losses were within one point, their lower bounds crossed the accuracy boundary. Thus neither selected threshold policy established the test retention criterion. This does not show that every threshold factor fails it. Figure 3 displays the full predeclared test grid descriptively; selection used validation data alone.

3.3 The original primary result remains inconclusive

In the original cohort, $\lambda = 0.1$ at $T = 16$ changed accuracy by -0.178 points (-0.600 to $+0.244$) and reduced total SynOps by an estimated 0.274% (-0.417 to $+0.965$%). Accuracy met the one-point criterion, but the operation interval crossed zero. The subsequent stronger-penalty findings do not replace that original primary outcome.

Original $\lambda = 1$ results were directionally consistent with the fresh cohort. At $T = 16$, the total-operation reduction versus baseline was 3.574% (3.122–4.025%), and accuracy changed by -0.156 points (-0.383 to $+0.071$). The selected fine-grid threshold factors were 1.30, 1.40, and 1.40 at $T = 8, 16, 32$. Regularization had higher accuracy and fewer total SynOps than these policies at every window (Appendix Table 3). These comparisons remain exploratory because they followed inspection of the same benchmark. We do not combine the two cohorts into a ten-seed estimate.

3.4 Unchanged input activity limits total savings

At 16 steps in the fresh cohort, baseline inference required approximately 500,287 SynOps per image, of which 460,558 came from input-to-hidden connections. Regularization reduced hidden-output SynOps from approximately 39,729

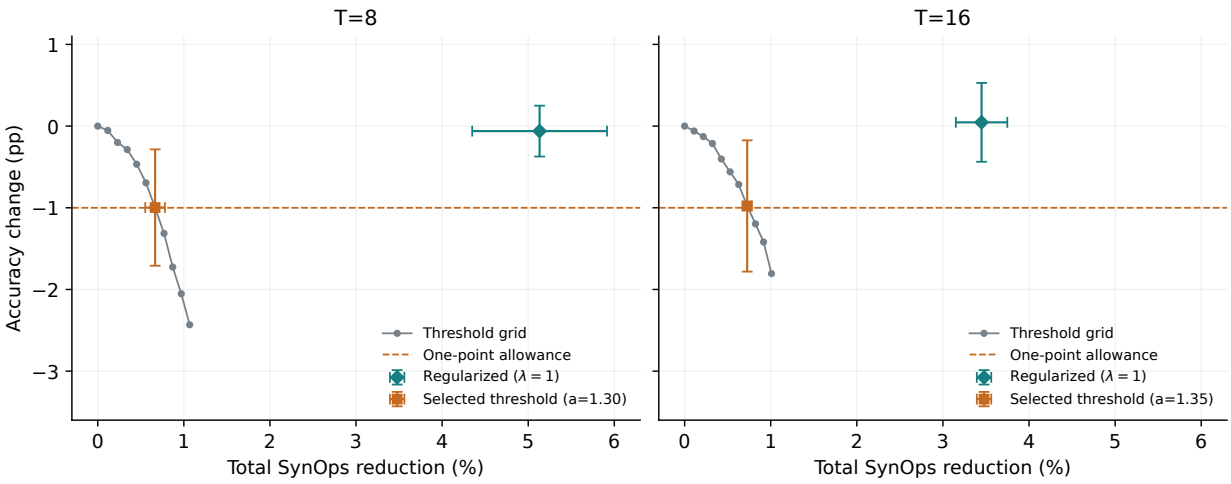


Figure 3: Fresh-cohort accuracy–activity trade-offs relative to baseline. Gray points follow the full predeclared threshold grid; connecting lines are visual guides. Orange squares identify validation-selected thresholds ($a = 1.30$ at $T = 8$, $a = 1.35$ at $T = 16$); teal diamonds identify regularization. Error bars show marginal unadjusted 95% paired intervals for the two selected policies, not a joint confidence region. The dashed line marks a one-point accuracy loss. No setting is selected from this test curve.

to 22,476 while leaving the matched input events unchanged. Total SynOps consequently remained approximately 483,034 per image (Figure 4).

The paired hidden-output reduction was 43.585% (38.159–49.011%), compared with 3.449% for total SynOps. Across the five fresh baseline seeds, the accounting ceiling in Eq. (11) averaged 7.939% at 16 steps, ranging from 7.537% to 8.680%. At eight steps it averaged 8.076%. Thus even complete suppression of hidden-output operations cannot yield a large whole-network reduction while this input pathway is fixed. The observed savings use only part of that accounting allowance because useful classification retains hidden activity. All 50 selected-policy/seed identities across the two cohorts agreed to numerical precision.

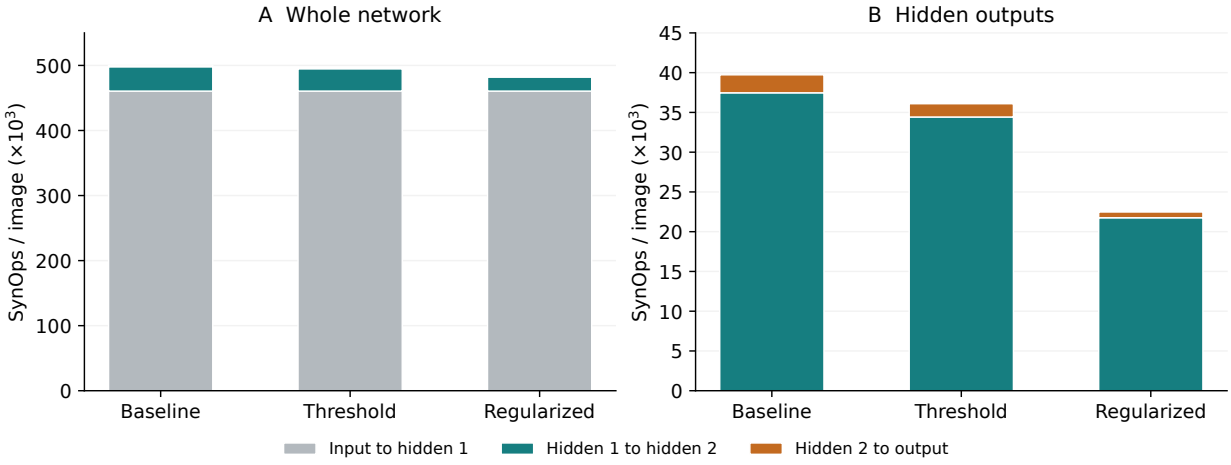


Figure 4: Fresh-cohort mean SynOps per image at $T = 16$. The input-to-hidden component (gray) is unchanged across matched policies; teal denotes hidden-1 to hidden-2 and orange hidden-2 to output. Panel B enlarges the two hidden-output components. Panel scales differ. Bars summarize five training seeds after averaging three encodings and illustrate the accounting, rather than confidence intervals.

3.5 Sensitivity to individual seeds and inference assumptions

A post hoc leave-one-seed-out check retained both primary criteria in all five four-seed subsets. The accuracy lower bounds ranged from -0.669 to -0.186 points, and total-reduction lower bounds from 2.994% to 3.256%. This checks dependence on a single run, not generalization to another dataset. An exact two-sided sign-flip check for the primary operation effect gives $p = 0.0625$: with five pairs there are only $2^5 = 32$ sign assignments. It assumes sign symmetry and is not distribution-free. This coarse diagnostic does not meet a 0.05 threshold, emphasizing that the declared positive result relies on the prespecified Student-t interval model. Both analyses were added after observing the results and do not replace the primary rule.

4 Discussion

The fresh-seed primary result answers the fixed question within the tested setting. A hidden-occupancy penalty of 1 retained accuracy within the declared allowance and reduced total effective SynOps. Its operating point also improved on the validation-selected uniform threshold policy in secondary comparisons at both windows. The effect on the whole network was modest, despite a substantial reduction in hidden-output operations.

Training under a penalty allows weights to adapt to an activity constraint. Increasing thresholds afterward changes the excitability of a representation learned at lower thresholds. This difference offers a plausible explanation for the observed trade-off, but the experiment does not isolate a mechanism. The penalty is not fanout weighted. Fanout-aware objectives already exist [5], while UniSparse changes both firing and connectivity [6]. We did not compare these stronger methods, layer-specific thresholds, or threshold-aware retraining; superiority over our selected control should not be extended to them. Nor do the methods have equal development costs: penalty search requires additional training, whereas threshold search reuses checkpoints. The reported inference counts exclude both search costs.

Input encoding is the principal accounting constraint. A dense 784-dimensional Bernoulli input continued to drive most counted operations after hidden activity declined. Other encodings or input connectivity could alter that balance, but their accuracy–activity trade-offs would require separate evaluation. A deployment claim would additionally need measurements or a hardware model that includes the costs omitted from effective SynOps. Fewer algorithmic events alone do not establish lower energy use.

The new cohort improves the evidence by fixing the stronger primary contrast before fresh training and evaluating five new seed pairs with different encoding banks. It does not provide an untouched benchmark or an external replication: the same Fashion-MNIST test images, split, architecture, and project pipeline were reused. The maximum encoding-bank window also changed from 32 to 16. Consistency across cohorts is informative, but their numerical effects should not be treated as exchangeable observations.

Five seeds leave limited information about the shape of the seed-effect distribution. The paired t intervals quantify training-run variability under the stated controls, not variability over datasets or splits. Secondary intervals do not account for validation selection or multiplicity. Although every seed benefits in operation count, the primary accuracy interval spans small losses and gains. The appropriate conclusion is retention within the one-point allowance, rather than equality or superiority of accuracy.

Finally, 15 epochs define a budget, not demonstrated convergence. Six of the 20 fresh models achieved their best validation accuracy at the final epoch, and 12 at either epoch 14 or 15. Longer training could change the relative operating points. A useful extension would fix a longer budget for a matched subset before execution. Testing a second architecture or dataset would then address transfer beyond this setting.

5 Conclusion

In this rate-encoded Fashion-MNIST SNN, the fixed fresh-seed comparison supports hidden-spike regularization as a way to reduce total effective SynOps while retaining accuracy within one percentage point. Regularization also improves on the tested validation-selected uniform threshold policy. The original weak-penalty primary result remains inconclusive for operation savings. Input events dominate the count, so a roughly 44% hidden-output reduction translates to only about 3.45% fewer whole-network operations at 16 steps. Assessing the full network, rather than hidden sparsity alone, is essential to interpreting this result.

Reproducibility and availability

The project repository is https://github.com/SakshamDev/snn-study. Reproduction scripts and numerical summaries accompany this manuscript; the archived evidence and environment are documented in the reproducibility guide. The accompanying reproduction script recomputes all 45 stored paired-effect summaries from seed-level data, verifies the input-cost identities, and produces the sensitivity diagnostics without training. Run `python reproduce.py` after installing the documented dependencies. This reproduces statistical analyses from supplied summaries; reproducing network outputs requires the checkpoints, dataset, and recorded environment. The separate raw-evidence audit verifies the 20-checkpoint return and all 720 evaluation records. The Kaggle notebook retains the exact 20-model training procedure. The recorded research environment used Python 3.13.15, PyTorch 2.11.0+cu128, torchvision 0.26.0+cu128, snnTorch 1.0.0, NumPy 2.1.3, and CUDA 12.8.

A Original-cohort selected-policy comparisons

Table 3 preserves the checkpoint follow-up as supporting evidence. The selected penalty was 1 at all windows. Threshold scaling reused baseline weights, and its factors were chosen using the finer validation grid. The intervals have the same seed-level definition as Eq. (14); these analyses remain secondary to the original weak-penalty primary contrast.

Table 3: Original-cohort selected regularization versus selected threshold scaling. Selected factors are $a = 1.30, 1.40, 1.40$ at $T = 8, 16, 32$, respectively. Entries are paired means [unadjusted 95% t intervals] over five seeds. These exploratory results are kept separate from the fresh cohort.

T	Accuracy change (pp)	Total SynOps reduction (%)	Hidden-output SynOps reduction (%)
8	0.798 [0.513, 1.083]	4.556 [3.742, 5.370]	60.980 [54.890, 67.069]
16	0.915 [0.211, 1.620]	2.761 [2.299, 3.223]	37.500 [32.580, 42.419]
32	1.036 [0.228, 1.844]	1.726 [1.048, 2.403]	22.129 [14.339, 29.918]

References

[1] Neftci EO, Mostafa H, Zenke F. Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. IEEE Signal Process Mag. 2019;36(6):51–63. doi:10.1109/MSP.2019.2931595.

[2] Eshraghian JK, Ward M, Neftci EO et al. Training spiking neural networks using lessons from deep learning. Proc IEEE. 2023;111(9):1016–1054. doi:10.1109/JPROC.2023.3308088.

[3] Narduzzi S, Bigdeli SA, Liu SC et al. Optimizing the consumption of spiking neural networks with activity regularization. In: ICASSP; 2022. p. 61–65. doi:10.1109/ICASSP43922.2022.9746375.

[4] Yan Y, Chu H, Jin Y et al. Backpropagation with sparsity regularization for spiking neural network learning. Front Neurosci. 2022;16:760298. doi:10.3389/fnins.2022.760298.

[5] Suetake K, Ushimaru T, Saiin R et al. Synaptic interaction penalty: Appropriate penalty term for energy-efficient spiking neural networks. Trans Mach Learn Res. 2024. https://openreview.net/forum?id=42BKnT2qW3.

[6] Shi X, Bu T, Yu Z. UniSparse: Combining weight pruning and spike sparsification in spiking neural networks. In: Proceedings of the 43rd International Conference on Machine Learning. Proc Mach Learn Res. 2026;306:111711–111725. https://proceedings.mlr.press/v306/shi26a.html.

[7] Stanojevic A, Woźniak S, Bellec G et al. High-performance deep spiking neural networks with 0.3 spikes per neuron. Nat Commun. 2024;15:6793. doi:10.1038/s41467-024-51110-5.

[8] Kim Y, Park H, Moitra A et al. Rate coding or direct coding: Which one is better for accurate, robust, and energy-efficient spiking neural networks? In: ICASSP; 2022. p. 71–75. doi:10.1109/ICASSP43922.2022.9747906.

[9] Yik J, Van den Berghe K, den Blanken D et al. The neurobench framework for benchmarking neuromorphic computing algorithms and systems. Nat Commun. 2025;16:1545. doi:10.1038/s41467-025-56739-4.

[10] Xiao H, Rasul K, Vollgraf R. Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms. arXiv:1708.07747; 2017. https://arxiv.org/abs/1708.07747.